

Software Architecture Document

HBR-QA Team

Kevin Guerra

Version 1.0

June 2015

Revision History

NOTE: *The revision history cycle begins once changes or enhancements are requested after the initial version of the Software Architecture Document has been completed.*

Date	Version	Description	Author
06/12/2015	1.0	Initial version	Kevin Guerra

Table of Contents

1. Introduction	1
1.1. Purpose	1
1.2. Scope	1
1.3. Definitions, Acronyms, and Abbreviations	1
1.4. References	2
1.5. Overview	2
2. Concept	2
3. Layers	2
3.1. Base, Framework Layer or Architectural Layer	3
3.2. High Level	4
3.3. Middle	4
3.4. Low Level	5
4. Structure	5
5. Initialization	5
6. Test Framework	5
7. Error Handling	6
8. Sample Test	6
9. Development	6

HBR-QA Testing Framework

1. Introduction

This document provides a high level overview and explains the main classes and interactions for a new framework and architecture for HBR-QA Testing. It could serve as an evolving document. It explains how a test developer will be able to use the system, how the system operates internally, including interactions with other libraries and frameworks, specifically vmtree.

NOTE: Development is at an early stage, therefore this document is subject to change.

1.1. Purpose

To provide a high level overview of the new framework, information on its usage and explain the similarities and differences with the current framework, so that developers who are already familiar with the previous framework would be better able to migrate.

1.2. Scope

Initially the scope is moderate, since this system is at the early development stage. It should explain the architectural layers, system initialization, output, configuration, object interactions, error handling, and main concepts.

There is much information that stems from the vmtree library, this framework will serve as a kind of wrapper that attempts to group functionality in a way that feels more natural, and an explanation on that will be provided. However, the internals on how those libraries work, are better detailed in their respective documentation. Therefore, only minimal information may be provided about them.

1.3. Definitions, Acronyms, and Abbreviations

The system will avoid abbreviations as much as possible, and only use those that the team members are accustomed to. In the new framework it will be necessary to change some terms in order to better model the system. This may cause some confusion at first, due to people being used to certain terminology. Nevertheless, the benefit of having a more accurate model far outweighs the disadvantages of having to relearn the terms, specially for those who are unfamiliar.

- **HBR** – Host Based Replication
- **HMS** - Host Management Server
- **VC** – Virtual Center
- **ESX** – Elastic Sky X ('The' Virtualization Server)
- **VM** – Virtual Machine
- **Layer 1** – High Level Layer, aka **Module**
- **Layer 2** – Middle Layer, aka **Object**
- **Layer 3** – Low Level Layer, aka **Support**
- **Base Layer** – Framework or Architectural Layer

Other terms may pop up, related to libraries, technologies, etc..., explanation on those is beyond scope.

1.4. References

- vmtree library

1.5. Overview

In order to fully document the aspects of the framework, the Software Architecture Document contains the following subsections.

Concept: Describes the system conceptually and its operation

Layers: Description about the multi-tier architecture, and how interactions happen

Initialization: How the framework, test and objects are configured, initialized and disposed of

Test Framework: Describes how classes from which a test object would derive

Error Handling: Best practices for error handling

Sample Test: A sample test scenario that describes the system in full operation.

Development: Methodologies used, approach.

2. Concept

- A test should interact with the system at the highest level possible
- A test configuration should contain if possible all data, so that the system can build the necessary objects, automatically
- When the system initializes, the test should already have all objects it needs
- The objects are initialized but not yet connected, unless specified in the configuration or through the framework
- Extra objects needed or configuration changes should be done via the framework API, or more precisely Layer 1 (High Level Layer)
- The framework is responsible for maintaining state and reporting results
- The test should follow a linear execution path as it currently does, thus avoiding threading issues and other complications

3. Layers

The layers contain classes which are instantiated as objects. Sometimes is challenging deciding in which layer an object should go. So the criteria we'll use is the following:

- Base: utilities, helpers, anything that is useful to all layers and test
- Low: Mostly stateless, and many will not be objects but functions. For example, parsing, file path handling, invoking certain functionality from libraries, where in those you pass in a context object, since the low level object is passed in a context, it need not keep state locally, more specifically, for instance, you invoke dd command from some host, however, you would not invoke the command from the middle layer directly, but an API should be available, so you would pass in an ssh connection object to the function and would run a high level command, say 'WriteBytes' done to a disk object, you don't care how it's done, but the low level function knows that it needs to invoke dd on the host of the connection you passed in.
- Middle: Objects that keep some state, but conceptually are of lower level, but support the higher layer. For example a connection class, state is necessary, but it is not suitable for a test to call directly into it, because the concern of a connection is of lower grade, IOW, a test should not have to be concerned about connectivity issues, such as initiating directly, etc...

- High: Objects here keep all the state provided by the configuration assigned at initialization. Ideally, tests only interact with this layer.

STATE

- Object state is encapsulated and managed internally. Data are **not** accessible outside, use proper API
- In the object hierarchy, state changes may affect contained objects, even parent objects (hopefully not.) Containers will use proper channels for effecting said changes
- Class data is part of state, for example fields, collections, etc...

PARAMETERS

- Parameter usage is minimized using a layered approach. Please see below for details
- Functions with one or two parameters can use positional parameters
- Functions with more, should use named parameters
- Where applicable, parameters can be passed as dictionaries, this is very common in python
- Sanitation (cleaning up, removing extra characters from strings, etc...,) verification (checking for the existence of a parameter,) validation, should be done preferably after init and in a module responsible for such tasks. The problem here is that, the module ends up 'knowing' too much about the system as a whole. However, the advantage is too good to pass up, and I've had very good success rates with this approach. The end result being that your code base ends up very uncluttered from these concerns. One way to think about this is akin to entry points to a function, where all of these are done. However in this approach, these tasks are done in one module. So, when you are working on your function, you make sure that all these preconditions are met in the proper module. However, because many function group naturally, the result is that the work done for one function should be applicable to the rest of the group. How does this work in practice? A simple example would be host configuration (host name, user, password.) During init, those values are checked, for example, that they exist, and that the data appears to make sense, then, when you actually call ssh connect and pass in the parameters, the function would not have to recheck them. This provides a good separation of concern about data validation, verification, etc... from its utilization.

3.1. Base, Framework Layer or Architectural Layer

- Configuration
 - Framework to build objects from configuration at initialization
 - Configuration file per test would allow for tailored setups, main configuration to contain 'Lego' blocks
 - Built objects are not to be initialized (meaning, creation of VMware objects or connections), unless a configuration parameter explicitly states so
 - Extra objects, override parameters, to be also processed at initialization, with the same effects as above
 - TO AVOID the HUGE MESS of configuration data: Configuration data only to be used at initialization, at which point the proper objects take on or appropriate the values, therefore, changing configuration data after initialization should have ZERO side effects
 - Configuration data are not state data, Configuration data are not state data, Configuration data are not state data... etc...
 - Configuration data are not parameters, Configuration data are not parameters, Configuration data are not parameters... etc...
 - The two eloquently put points above avoid a lot of misunderstandings about what's happening in the code, this is major
 - Configuration data should be fairly stable, that is, it should resemble the objects capabilities, and only change to reflect changes, additions or removal of capabilities, this implies that a good location to put extra data, would not be configuration data

- State data is the responsibility of each object and should not be made available directly, only through proper APIs, ideally the objects should use FSM (Finite State Machine) where applicable
- We should never treat an object as another via injecting configuration data, as it is done now. A properly constructed class instance should be fairly stable. If you need an object of the same type, but with different configuration, for the love of Pete, just create another object, there is immense confusion about how to interpret object interactions because of this
- Because of how we keep proper configuration data, parameter passing should be minimized. This alone would result in a reduction of code of about 40%, it's hard to believe, but notice that in the current code base about that percentage is dedicated to handling of configuration/parameter/state data, along with verification, parsing, etc... which happens at various stages for the same data!
- After initialization, all objects have been initialized with their proper data, at this point the Configuration object may remain, but its use should be extremely rare, if at all. The passing of data between objects should be done through the proper API, we should never use sneaky ways to pass data around or even return values. Python can return tuples or even dictionaries. The runtime overhead is negligible comparing with the huge cognitive overhead of sneaky data passing
- **Logger**
 - Automatic logging for functions
 - Colorized
- **Report**

3.2. High Level

- **Actors:** Configured objects, will contain all objects listed in configuration
- **Group:** Hbr group, it may morph into something that will manage objects more naturally, or it could simply expand its capacity, so that, if you group objects, you may call high level functions on the group and it would affect the group members where applicable
- **Esx**
- **Hbr**
- **Vc**
- **Vm**
- **Replica**

3.3. Middle

- **Device**
- **Disk**
- **Nic**
- **Connection**
- **Command?**

3.4. Low Level

Layer 3 must do the heavy lifting, path cleaning, parsing, etc... because these should be low grade concerns and we want to avoid cluttering the higher layers and specially the tests

- **SshConnection:** used in vmtree and gemini
- **SoapConnection:** used in vmtree
- **Staf**
- **Spaz**
- **RemoteCommand**

LocalCommand

4. Structure

Directory structure and file organization

Gemini

- Suite

- FunctionalGroup1, etc...: Test1, Test2, etc...

CheckConfiguration: Check server directories, configs, etc... explicit call, not done for all tests, time consuming

- Test

- Lib: Log, Configuration, Report,

- Module: Actors, Group, Esx, Hbr, Machine, Replica,

- Object: Device, Disk, Nic, Connection, Command,

- Support: SshConnection, SoapConnection, Staf, Spaz, RemoteCommand, LocalCommand,

- Setup: Factories, Configure, Initialize, Verify, Cleanup

5. Initialization

- User runs a test from the command line

- Configuration files are read:

- Global configuration: contains configuration blocks, and 'default' virtual system, for example: ESX, Virtual machines, Replicas and such should not be here

- Test configuration: if exists, contains extra blocks, other machines, etc... data on which machines to startup

- User configuration: contains specific user configuration, such as IP addresses, host names, etc... the values here replace those in the other configuration files, IOW, override

- Objects get created from configuration and added hierarchically in Actors

- Automatic commands in configuration are run, such as connect to ESX, create VMs, etc...

6. Test Framework

- After initialization, the test code can call high level commands, with reasonable expectations that the 'right' things will happen:

- Error reporting, to log and if applicable to Report

- When reasonable, error recovery, for instance if an ssh connection is broken, it can be retried a max_num of times

- Derives from Python unittest, so it's predictable, we'll use the standard assert facilities

7. Error Handling

- Because this is a testing framework, a proper error handling strategy is integral, however, the aim should be to handle it consistently and ideally automatically by the Layer 1 classes

- Typically in most API frameworks, error handling responsibility is delegated to the caller or client. However, I believe in a testing framework, the framework itself is not as much an API as it is an actual system. Therefore, this responsibility would be more appropriately centralized and handled in the framework, for the most part.

- The tests should have some asserts and verification lines, however, whenever possible these should happen in Layer 1

- This would have the effect that the test code would be very minimalistic, since having a good separation of concerns is paramount to good OOP. For example, if you call a Layer 1 function to say VM.PowerOn(), it should **not** be the responsibility of the test to make sure it happened, but of the Layer 1 object, moreover, the framework should also handle the error
- For the reason above, the error handling for that function should of course be in Layer 1. The effect would be that when an error occurs, by default they would abort the test, as the current framework does now. However, flexibility should be built so that when desirable, some methods would report an error but not abort the test, so it should be up to the test developer to opt out of the default
- Exception handling seems to be quite adequate for error handling in most cases, however, we should minimize the proliferation of derived exception classes. The best practice is to use what the language and system provide, and only create a new class when its use would offer additional functionality other than just print out the error. In other words, when we want to catch a very specific type of error because we will handle it in a non-standard way (more than print.) A good example would be for an SshConnection object. Let's say that a connection gets broken, an operation is sent through the broken pipe, bam! Error, we would have a notification method that would get called in order to re-establish the connection. Or not, simply abort the test, since a broken connection is usually an indication of something bad not usually related to our test. This is a good illustration of why extra exception classes are really, exceptional

8. Sample Test



9. Development

PROGRAMMING STYLE

- ✓ **OOP:** I'll attempt to follow most/all OOP principles, unless impractical. I like the [SOLID](#) guidelines:
 - ❖ [Single responsibility principle](#): A class should have only a single responsibility. This one is hard to reason about in the higher layers, but should apply conceptually
 - ❖ [Open/closed principle](#): Open for extension, closed for modification. This can be achieved but only completely after the framework is done. However, because of layering, changes on lower layers shouldn't disturb the tests so much.
 - ❖ [Liskov substitution principle](#): Polymorphic behavior, permits use of mock objects
 - ❖ [Interface segregation principle](#): Currently we have the opposite of this. We have several god classes. Some people opine that OO code is hard to read, because of the levels of indirection in objects calling other objects. This is compounded by this principle because ideally you have more and smaller objects that interact. I think that the advantages far outweigh the disadvantages. This, in and of itself allows for good code orthogonality, which allows you to do more with less code, and the less code you have, the easier it is to understand. This does not mean that one can just look at a framework and just understand it right away. There is a learning curve as with anything of relative complexity. So practice makes perfect.
 - ❖ [Dependency inversion principle](#): This one is a bit clunky to do in Python, but it's doable. This would give us the ability to inject mock objects to the factory classes, so that we can tests our tests while we develop. We should prepare to say good bye to having to run the full test to test our newly written or changed code. We can also do negative testing with mock objects. For example, you can pass in a parameter, or whatever to cause the mock object to produce an error, and we would see that our test can react properly.

- ✓ **Functional Programming:** I'll attempt to code methods with no variables or state where applicable. This has some advantages in code understandability and is less error prone. Some code should be just a function. People get caught up in the conundrum Java created when they sort of dictated everything should be a class. Well, not so at all. Some concepts map naturally to functions.
- ✓ **Simplicity over cleverness:** Self explanatory
- ✓ **YAGNI:** Keep things as simple as possible and only add code when needed, however, I'll try to keep some symmetry, for example, if you have a create, put a delete, even if at the moment is not needed and adding it would not take too long
- ✓ **Orthogonal Design:** [DRY](#) principle, good refactoring, avoiding duplication of code, composition
- ✓ **Naming Convention:** This is, I think a very important part
 - ❖ Eliminate abbreviations, because they usually cause ambiguity, are used inconsistently and make the code very ugly. With exceptions for VMWare objects: VM, ESX, HBR, and may be a few others. However, if we use these, when choosing method names, they would morph into Vm, Esx, Hbr, etc...same for variable names. On second thought I think the V in Vm is superfluous because we only deal with virtual machines. So I think just the word Machine, and then you can inherit from that ProtectedMachine and ReplicaMachine, or rather ReplicatedMachine, protected is asymmetrical, because one is Replicated and the other a Replica, yes it is by implication 'protected' but in reality it is more of a replica source. Sorry, I have to work on the name or maybe protected should suffice.
 - ❖ Eliminate superfluous prefixes and suffixes. They do not convey any information and have the effect that they make you pause when reading the code. For example, the libraries of the form HbrApiFileName.py: We already know it's an API, we already know it's about HBR because the parent directory is named to allude to that fact. Keeping simpler names makes it easier to 'guess' what module or method one needs, when one is not familiar enough with the code.
 - ❖ Names like Helper, Utility, Manager, Module, Framework, Api, those aren't very useful. They're at least bad style.
 - ❖ Careful and consistent selection of words. If we choose delete() all should be named that, not remove(), erase(), etc...
 - ❖ Creating a coding or naming convention is beyond scope at this point, but there are a few out there in common use: [PEP](#), [Google](#), [Pythonista](#). I'm not very picky as to what we use (except for names) but we should all agree on something we can all live with.
- Question: Do we have a python coding standard at VMWare?**
- ✓ **Decorators:** Python decorators are very handy because they allow for good separation of concerns. I'm already using them for logging, at function start and exit. Planning on using them for parameter/input validation in applicable cases.